Activation-Aware Merging of LoRA Adapters for Multi-Task Learning

A controlled study of similarity, functional objectives, and routing

COMP9501 Advanced Machine Learning
Group 13

11 November 2026

Abstract

Combining task-specific low-rank adaptations can reduce the need to manage distinct weight states, but averaging updates can introduce interference. We investigate whether functional information can predict or mitigate this interference in a small, reproducible multi-task setting. Four Qwen2.5-0.5B adapters are trained on SST-2, MRPC, RTE, and CoLA. A content-grouped five-way protocol separates teacher training, teacher checkpoint selection, merge fitting, merge selection, and final evaluation. Across three training seeds, learned layer-wise coefficients achieve a mean native task score of 0.7132 with cross-entropy (CE), compared with 0.5578 for averaging and 0.6146 for our TIES baseline. Teacher-logit KL reaches 0.6987 and does not outperform the matched-capacity CE control on average. Our initial unit-weight extension tests hidden-response matching, shuffled-response controls, and performance-oriented routing. The exploratory hidden-response and routing controls give paired effects of -0.0733 native score versus FP32 CE and -6.93 accuracy points versus task-ID routing, respectively. A further fixed-grid sensitivity and conservative-routing study evaluates a larger, exact-tuple-cleaned official-validation split. Selected hidden versus CE changes native score by +0.0055; the benefit gate changes macro accuracy versus task ID by +0.00 points, with exploratory scope. The study identifies the distinction between source recognition and useful expert selection, and measures routing overhead on an A100. A direct CSA extension additionally tests structured compression of expert-response matrices with full basis accounting. Its contribution is a controlled implementation and analysis of the original hypothesis; it does not establish a new state-of-the-art merging algorithm or robust out-of-domain generalization.

1 Introduction and research questions

LoRA adapts a pretrained weight matrix using a low-rank update while retaining a shared backbone [2]. A collection of task adapters therefore offers a practical multi-task system. A merged model can offer a single backbone weight state, but combining independently trained updates can damage individual task performance. Task Arithmetic, TIES, and DARE provide weight-space approaches to this problem [3, 4, 5].

Our original proposal hypothesized that adapters with similar activations can be merged safely, while conflicting inputs or tasks require dynamic gating. We translate this broad hypothesis into three questions:

1. **RQ1:** Does activation similarity reliably predict the performance loss of a task-pair merge?
2. **RQ2:** Does functional supervision improve a static merge after controlling parameter capacity, optimization, precision, and access to labels?
3. **RQ3:** Can an input-dependent router select useful experts beyond the directly available task-ID baseline, and what computational cost does this introduce?

We study one small backbone and four binary tasks to permit explicit data and numerical checks. We retain negative results, distinguish exploratory stages, and compare learned coefficients against both fixed weight-space rules and supervised controls. The resulting contribution is a proposal-aligned empirical investigation with reproducible artifacts, an explicit information protocol, and diagnostics of misleading proxy metrics.

The original deployment motivation also requires qualification. Our actual reference system shares one base model and stores four small adapters; it does not duplicate four full backbones. Consequently, a single merged weight state is not automatically a substantial total-storage saving. This distinction matters when judging the practical value of static merging and routing.

2 Related work and positioning

Task Arithmetic represents adaptation as displacement from a common pretrained model and combines these task vectors [3]. TIES addresses small or conflicting parameter changes through trimming, sign election, and disjoint merging [4]. DARE randomly drops and rescales updates before combination [5]. We implement explicit variants and use a fixed candidate grid; comparison with these implementations is not a claim to reproduce every official benchmark setting.

Functional information has established precedents. RegMean uses layer-input Gram statistics to match linear-layer responses [6]. AdaMerging learns task-wise or layer-wise coefficients using prediction-entropy adaptation [7]. KnOTS studies representation alignment in LoRA merging and uses a shared SVD space [8]. Hidden-representation supervision also predates this project in knowledge distillation, including FitNets [9]. These works motivate our tests and constrain novelty claims: layer-wise coefficients, response matching, and low-rank alignment are not inventions of this project.

Our implementation combines a constrained adapter-span parameterization with controlled comparisons of CE, output KL, hidden-response objectives, and routing targets. Unlike the entropy objective of AdaMerging, our coefficient selection uses labels from a declared selection partition. Unlike RegMean and KnOTS, we do not solve full layer-weight regression or perform shared-SVD alignment. Our evidence concerns this implementation and task-aware protocol, not superiority over those complete methods.

3 Model and merging methods

3.1 Adapters, heads, and the task-aware protocol

For an adapted layer ℓ, task t supplies

$$\Delta W_{t,\ell} = \frac{\alpha}{r} B_{t,\ell} A_{t,\ell}, \qquad W_{t,\ell} = W_{0,\ell} + \Delta W_{t,\ell}. \tag{1}$$

We use $r = 8$, $\alpha = 16$, and the query/value projections of Qwen2.5-0.5B [11]. Each teacher trains its LoRA factors and a task-specific classification head; the pretrained backbone remains frozen. Every evaluation method receives the same known task ID to select the corresponding frozen head. The four binary tasks have different label meanings, so a common output dimension is not a common label space.

We combine materialized BA updates, rather than averaging A and B separately: separate factor averaging introduces cross-adapter products. The sum of four rank-eight updates can have rank up to 32, subject to matrix dimensions and dependencies. Our final static model is a materialized backbone; we do not claim it is a single rank-eight adapter.

3.2 Static coefficient parameterization

For each layer, the merge is

$$W_{m,\ell} = W_{0,\ell} + g \sum_{t=1}^{4} p_{\ell,t} \Delta W_{t,\ell}, \tag{2}$$

$$p_{\ell,t} = \mathrm{softmax}(a_\ell)_t, \qquad g = 0.5 + 3.5\,\sigma(b). \tag{3}$$

The layer-wise variant stores $24 \times 4 + 1 = 97$ trainable scalars; a global vector stores five. Softmax introduces a shift redundancy, so these are tensor parameter counts, not independent degrees of freedom. Within a layer, query and value updates share the task mixture. Base weights, adapter factors, and task heads remain frozen during coefficient fitting.

The regularizer penalizes deviation from uniform mixture weights and from gain two:

$$R = \frac{1}{L} \sum_{\ell} \sum_{t} (p_{\ell,t} - 1/4)^2 + \log^2(g/2). \tag{4}$$

Its weight is 0.01. Fitting evaluates the exact low-rank response of the weighted update. Deployment materializes the selected ΔW into one backbone.

3.3 CE and teacher-output KL

The CE control optimizes ground-truth classification loss. The output-matching variant minimizes

$$\mathcal{L}_{\mathrm{KL}} = T^2\, D_{\mathrm{KL}}(\mathrm{softmax}(z_t/T) \,\|\, \mathrm{softmax}(z_m/T))\,, \quad T = 2, \tag{5}$$

where z_t is the frozen individual teacher's output for its own task. Both variants use the same coefficient capacity, initialization, update count, sample order, and label-based checkpoint-selection access. KL does not use fit labels in its gradient target, but teacher training and merge selection use labels. The complete pipeline is therefore supervised. Output KL also does not directly test an intermediate-activation similarity hypothesis.

3.4 Task-conditioned local-response prototype

Our earlier static prototype uses linear adapter responses to connect the proposal's similarity idea with a fitting objective. On task-t calibration inputs, we collect each adapted module's input $x_\ell^{(t)}$ along the individual teacher's path and compute every adapter response $u_{j,\ell} = \Delta W_{j,\ell} x_\ell^{(t)}$. A constrained quadratic fit minimizes the mismatch between $\sum_j p_{\ell,j} u_{j,\ell}$ and the own-teacher response $u_{t,\ell}$, with simplex weights and optional shrinkage toward uniform. Response Gram matrices are normalized by output dimension and valid-token count; the global variant additionally normalizes block traces. Gain is selected separately.

This is a response-statistics method restricted to the adapter span, related to the output-preservation motivation of RegMean. It is not RegMean's full weight-matrix solution. Using identical inputs and equally weighted targets also admits uniform averaging as an optimum, so task-conditioned paths provide the additional information. Lowering a local quadratic surrogate does not guarantee the behavior of the nonlinear materialized network. We retain this prototype as exploratory protocol-v2 evidence.

3.5 Direct hidden-response objectives and controls

Let $h_t(x)$ be the attention-mask token mean of the final hidden layer, and let $h_0(x)$ be the corresponding base response. Define $d_t = h_t - h_0$ and $d_m = h_m - h_0$. The new hidden objective is

$$\mathcal{L}_H = \frac{1}{N} \sum_i \frac{D^{-1} \sum_j (d_{m,ij} - d_{t,ij})^2}{\max\{D^{-1} \sum_j d_{t,ij}^2, 10^{-4}\}}. \tag{6}$$

Subtracting the same base cancels in the numerator: the distinctive normalization uses teacher update energy, not a new distance identity. This mean-pooling proxy also differs from the model's native classification pooling, so small hidden loss does not guarantee preserved class decisions.

We compare five FP32 profiles: layer CE, layer hidden-only, layer CE+H, layer CE+shuffled H, and global hidden-only. The combined hidden weight is fixed at one. The shuffled control applies a fixed within-task derangement to teacher deltas: it retains the target set while destroying same-input correspondence. This tests whether pairing carries information beyond the response distribution. It does not remove every source-style cue. The FP32 CE control isolates loss changes within the extension; historical BF16 CE/KL rows remain separately identified.

3.6 Input-dependent routing and oracle analysis

A source router predicts the input dataset using frozen base mean features. Our extension also fits a performance-oriented router. For each fit input with label y, its target is the expert minimizing ground-truth negative log-likelihood under the known task head. Its features concatenate standardized base responses and the already available task one-hot vector. This fitting requires fit labels and offline evaluation of all experts.

Both routers select logistic-regression $C \in \{0.1, 1, 10\}$ using the same merge-selection task metric. We compare them with direct task-ID selection and seeded random routing. An analysis-only oracle selects the final-set expert with highest true-class probability. It gives an accuracy ceiling for this bank/head protocol, using final labels, and cannot be deployed. Agreement with its minimum-NLL expert identity is distinct from task accuracy.

3.7 Loss-weight sensitivity and conservative gates

The unit hidden-loss weight is a fixed design choice, not an established optimum. A follow-up grid uses $\lambda \in \{0, 0.01, 0.1, 1\}$ with the same 97 scalars, FP32, initialization, update count and sampled examples. We reuse the completed zero (CE) and unit-weight endpoints and fit the two intermediate weights for aligned and shuffled targets. Each lambda first selects its step on the original merge-selection role; the grid then selects lambda by that role's native metric, preferring smaller lambda on ties. Aligned and shuffled grids have equal candidate budgets, but both offer more selection candidates than the single CE trajectory.

A confidence gate reuses the frozen expert-ID classifier and switches only above maximum-probability thresholds $0, 0.5, 0.7, 0.9$, or chooses always-own. A benefit gate instead fits four Ridge regressions to fit-set correctness differences,

$$y_j(x) = \mathbf{1}[\hat{y}_j(x) = y] - \mathbf{1}[\hat{y}_{\text{own}}(x) = y]. \tag{7}$$

Its inputs are standardized base features and known task one-hot. Ridge strengths are $1, 100, 10000$; predicted-gain thresholds are $0, 0.02, 0.05, 0.1$. The own expert has fixed reference gain zero. These are uncalibrated regression scores, not probabilities of benefit. Fitting uses class labels and all experts' offline predictions; its capacity and information budget differ from static coefficient fitting.

Both gate families include always-own. Eligible selection candidates must have native score and macro accuracy at least task ID, and worst-task accuracy no more than 0.01 lower. We then

maximize native score, macro accuracy, and finally minimize switching; remaining ties retain the baseline. These constraints concern selection data only and do not guarantee final safety. Serialized scalers and coefficients are reloaded and checked against the in-memory routes. An always-own choice can bypass feature extraction at serving, but represents rejection of the new router rather than a new routing gain.

The combiner uses only selection fields to choose lambda and gates, then writes choices before reading cached final prediction arrays or loading official validation. Earlier final metrics already coexist in run files and had informed this follow-up design; this order does not create a pristine-test claim.

3.8 A larger, content-filtered evaluation split

We additionally evaluate pinned GLUE official validation after selection is locked. We remove normalized complete text tuples found in *any original training example* across the four tasks, then deduplicate tuples within each validation task. Filtering uses text only, not labels. Exact IDs, content hashes, fingerprints and removal counts are recorded. Near duplicates, sentence reuse within different pairs and pretraining contamination remain possible. Official validation was explored historically with earlier teachers, so this is a larger separate-split check, not a new blind benchmark or proof of OOD robustness.

The locked comparison includes task ID, source/expert routers, the two gates, GT oracle, FP32 CE, selected aligned/shuffled grids, and the original v3 selection-locked Average/TA/TIES/DARE variants. Tasks have different retained sample counts: accuracy is macro-averaged over tasks, whereas switching fraction counts individual inputs. Three teacher seeds still give three replications; repeated coefficient fits do not increase this number.

4 Experimental protocol

4.1 Data and information boundaries

We use SST-2, MRPC, RTE, and CoLA from GLUE [10]. Protocol v3 splits the original training data into five content-grouped roles. Normalized complete field tuples receive a fixed hash-based role independent of class labels; duplicates share a role, including across tasks. One representative per normalized tuple is retained. Dataset revision, source fingerprint, IDs, and manifest SHA are saved.

Table 1: Protocol v3 sample counts per task. Final is an internal partition of original train, not official GLUE validation.

Task	Train	Teacher select	Merge fit	Merge select	Final
SST-2	4096	128	128	128	256
MRPC	2391	128	128	128	256
RTE	1571	128	128	128	256
CoLA	4096	128	128	128	256

All 12 teachers are freshly trained under these boundaries. Re-splitting an old calibration pool would not undo prior teacher checkpoint selection, so old teachers are not reused for v3. Exact-tuple grouping does not remove near duplicates, repeated individual sentences within different pairs, or possible pretraining contamination. Earlier GLUE exploration also informed the research. The new hidden/routing extension was designed after seeing v3 final results and is explicitly exploratory; per-run fit/select separation does not restore pristine-test status.

4.2 Training, selection, and comparators

Teachers use three epochs, batch 16, maximum length 128, LoRA dropout 0.05, adapter learning rate $2 \cdot 10^{-4}$ and head rate 10^{-3}. Teacher selection minimizes loss on its dedicated role. Training seeds are 42, 43, and 44; the pretrained backbone and common head-initialization seed are fixed. These seeds quantify some training variability on one shared partition, not independent dataset replications.

Coefficient profiles use Adam, 120 updates, batch eight, learning rate 0.03, and checkpoint candidates $\{0, 30, 60, 90, 120\}$. Step zero is included; ties keep the earlier step. Selection maximizes the macro mean of SST-2/RTE accuracy, MRPC (accuracy + F_1)/2, and CoLA MCC. We call this the *native task score*; it is not the full official GLUE score. We additionally report mean accuracy, worst-task accuracy, and the drop relative to the same-seed individual teacher.

The extension uses the allocated NVIDIA L4 after A100 availability could not be obtained. Its new FP32 controls share the same device and precision. Historical v3 teacher training, BF16 coefficient fits, and the separate serving-cost benchmark used A100. We keep these stages distinct and check restored teacher task-ID performance against the saved reference; any numerical runtime difference is recorded rather than treated as a new training outcome.

Weight-space comparators use a fixed 17-candidate grid: averaging; TA scales 0.25/0.5/1; per-matrix TIES retention 0.2/0.5 and scale 0.5/1; DARE retention 0.9/0.5/0.1 and TA scale 0.25/0.5/1. DARE mask seed stays 42. Four-task averaging equals TA scale 0.25. Selection samples are shared, but candidate counts, fit supervision, and optimization cost differ; this is not an equal-total-information or equal-compute comparison.

Each run writes selected choices before loading its final role. The extension preserves this execution order while disclosing prior final-set inspection. All tables report three-seed means and sample standard deviations. With $n = 3$, we show paired outcomes and avoid significance claims.

5 Results

5.1 RQ1: similarity is an unstable merge-safety proxy

Our earlier protocol-v2 diagnostic evaluated the six equal-weight pairs of four tasks. Spearman correlations between delta-activation cosine and mean pair accuracy loss were -0.6571, -0.4857, and $+0.0857$ across three training seeds. Delta similarity was more negatively associated than raw similarity in only one seed. Six pairs share adapters, so the 18 pair/seed observations are not independent experiments.

This exploratory evidence does not establish a reliable safety threshold. Raw similarity can be dominated by the common pretrained model, while subtracting a base response makes the signal more update-specific without ensuring that it predicts nonlinear merged-model quality. Protocol-v2 pair diagnostics and protocol-v3 final scores are presented as separate stages rather than pooled into one confirmatory result.

The protocol-v2 local-response fit similarly failed to improve its uniform reference: global/layer native means were approximately 0.6226/0.6227, against 0.6365 for the selected uniform gain. This motivates testing end-to-end objectives and their controls rather than treating response-Gram optimization as sufficient evidence of a useful merge.

5.2 Protocol-v3 main comparison

Table 2: Frozen protocol-v3 results. Three training seeds; mean and sample SD. Native score is the macro task-specific metric, not official GLUE.

Method	Native score	Mean accuracy	Worst accuracy
Individual	0.7695 ± 0.0114	0.8252 ± 0.0083	0.7487 ± 0.0137
Average	0.5578 ± 0.0246	0.7080 ± 0.0068	0.5430 ± 0.0377
Task Arithmetic	0.5954 ± 0.0211	0.7152 ± 0.0132	0.5872 ± 0.0148
TIES (our variant)	0.6146 ± 0.0240	0.7152 ± 0.0096	0.5378 ± 0.0293
DARE+TA	0.6065 ± 0.0287	0.7077 ± 0.0131	0.5443 ± 0.0482
Uniform, selected gain	0.5889 ± 0.0256	0.7223 ± 0.0069	0.5768 ± 0.0328
Layer CE (v3 BF16 fit)	0.7132 ± 0.0225	0.7848 ± 0.0031	0.6875 ± 0.0141
Layer KL (v3 BF16 fit)	0.6987 ± 0.0080	0.7829 ± 0.0073	0.6719 ± 0.0273

The v3 results show substantial interference under averaging. Learned layer coefficients improve the fixed weight-space grid, but the matched-capacity CE control exceeds teacher-output KL in mean native task score. Their paired KL-minus-CE differences are +0.014052, −0.045911, and −0.011545, averaging −0.014468. Thus the observed improvement over weight rules cannot be uniquely assigned to the teacher-output target.

Individual remains the available task-aware quality reference. CE mean accuracy is 0.7848 versus 0.8252 for individual; worst-task accuracy is 0.6875 versus 0.7487. CE's fit labels and additional optimization also distinguish it from the weight rules, so improvement over TIES is an empirical pipeline comparison rather than a pure capacity-independent activation effect.

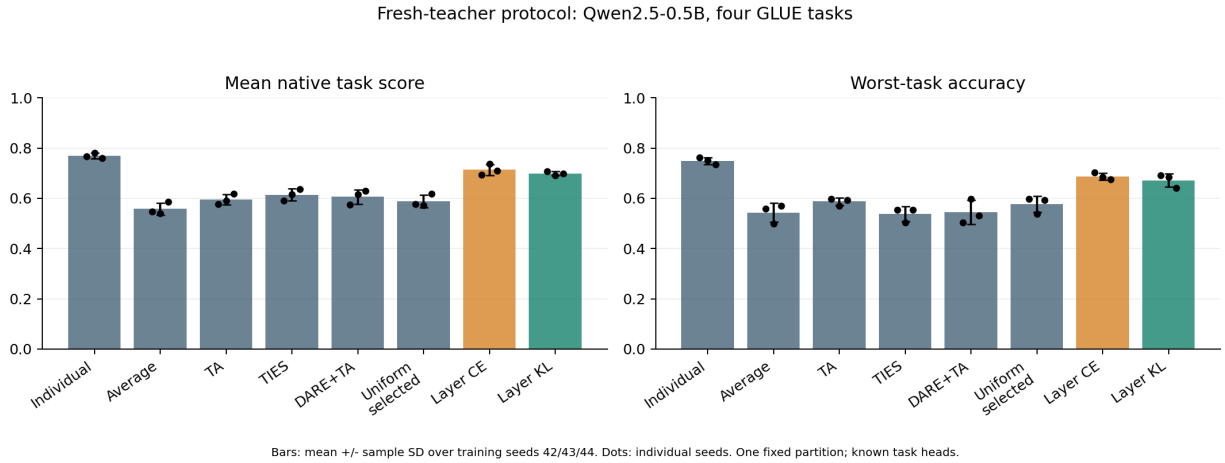


Figure 1: Frozen v3 baseline study: means, sample SD, and individual training seeds. Native metric definitions differ by task; worst-task accuracy is computed per seed before aggregation.

5.3 RQ2: direct activation controls

Table 3: Exploratory direct-activation controls. All FP32. Layer methods have 97 scalars; global has five.

Method	Native score	Mean accuracy	Worst accuracy
Layer CE (FP32)	0.7130 ± 0.0211	0.7861 ± 0.0029	0.6901 ± 0.0137
Layer hidden	0.5901 ± 0.0259	0.7285 ± 0.0010	0.5820 ± 0.0170
Layer CE + hidden	0.6397 ± 0.0281	0.7643 ± 0.0182	0.6927 ± 0.0407
Layer CE + shuffled	0.6195 ± 0.0056	0.7510 ± 0.0195	0.6641 ± 0.0629
Global hidden	0.5841 ± 0.0297	0.7298 ± 0.0166	0.6055 ± 0.0768

The paired native-score difference of CE+hidden minus same-precision CE is -0.073280, positive in 0/3 seeds. Against the shuffled-response control the mean difference is +0.020254, positive in 2/3 seeds. The aligned response penalty does not add task-quality benefit over CE in this setting. These descriptive comparisons isolate the selected response-pairing and capacity controls; the sample is too small and the evaluation history too exploratory for a confirmatory claim.

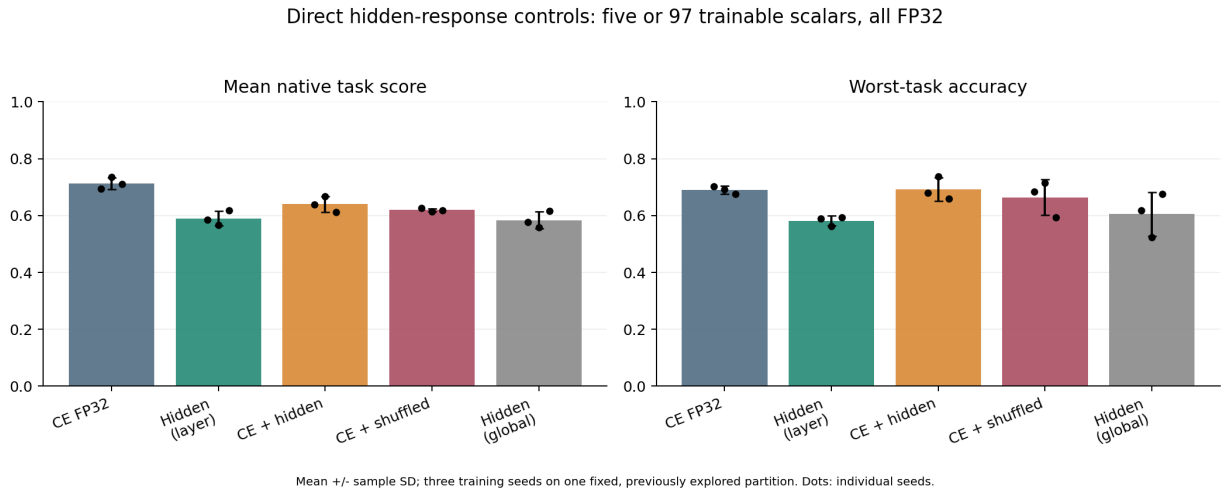


Figure 2: Exploratory FP32 activation controls. CE+H and CE+shuffled H share parameter count and update schedule. Global hidden-only has five scalars; the layer variants have 97. The fixed v3 final partition had already informed this extension's design.

The selected coefficient checkpoints are also evaluated on the aligned hidden-fit objective. This is a diagnostic, not an additional selection criterion. Reduction of a response surrogate is meaningful only alongside task quality and the shuffled control; optimizing a representation proxy can preserve irrelevant features or conflict with classification requirements.

5.4 Follow-up: is unit hidden weight too strong?

Table 4: Fixed lambda sensitivity. Same precision/capacity; three teacher seeds. Zero is CE. Lambda grid increases selection budget.

λ	Aligned select	Shuffle select	Aligned final	Shuffle final
0.0	0.6870 ± 0.0444	0.6870 ± 0.0444	0.7130 ± 0.0211	0.7130 ± 0.0211
0.01	0.6835 ± 0.0346	0.6832 ± 0.0356	0.7182 ± 0.0158	0.7199 ± 0.0141
0.1	0.6778 ± 0.0345	0.6752 ± 0.0358	0.7029 ± 0.0363	0.6854 ± 0.0436
1.0	0.6109 ± 0.0097	0.6051 ± 0.0194	0.6397 ± 0.0281	0.6195 ± 0.0056

Selection chooses zero hidden weight in 2/3 seeds. The selected aligned-grid minus CE native-score difference is +0.007764 on internal final and +0.005542 on cleaned official validation. These are pipeline comparisons with additional selection candidates, not isolated evidence of a unique functional effect.

Both families select $\lambda = 0.01$ for seed 42 and CE ($\lambda = 0$) for seeds 43 and 44. On cleaned official validation, the shuffled grid is descriptively slightly higher than the aligned grid (0.7107 versus 0.7106). Thus the small grid-selected gain over CE does not isolate a benefit of correct input-response correspondence; two seeds use no hidden target at all. Generic regularization or additional selection opportunities are possible explanations, not mechanisms established by these data.

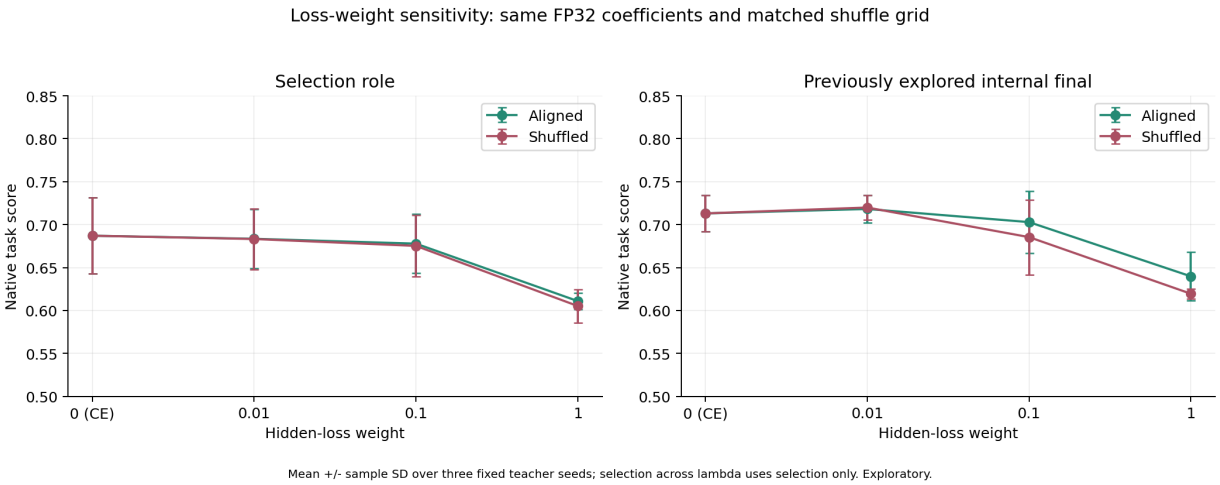


Figure 3: Predeclared lambda sensitivity. Selection and previously explored internal-final metrics are shown separately. Aligned and shuffled grids share budgets; zero is the CE reference, not a functional-method improvement.

5.5 RQ3: routing useful experts

Table 5: Exploratory task-aware routing. Oracle is a GT-assisted accuracy ceiling; native scores are not claimed to be upper bounds.

Method	Native score	Mean accuracy	Worst accuracy
Task ID / own expert	0.7695 ± 0.0114	0.8252 ± 0.0083	0.7487 ± 0.0137
Source router	0.7683 ± 0.0119	0.8232 ± 0.0083	0.7448 ± 0.0137
Expert router	0.6559 ± 0.0098	0.7559 ± 0.0051	0.6497 ± 0.0239
Random	0.5787 ± 0.0298	0.6875 ± 0.0220	0.5781 ± 0.0295
Oracle (final GT)	0.8974 ± 0.0028	0.9255 ± 0.0025	0.8685 ± 0.0090

The expert router changes mean accuracy relative to task-ID selection by -6.934 percentage points, improving in 0/3 seeds. The GT oracle increases mean accuracy by 10.026 points over task ID. This gap measures bank-level headroom under impossible label access, not a realizable routing gain. Our performance-oriented router fails to realize that headroom and is worse than direct task-ID selection.

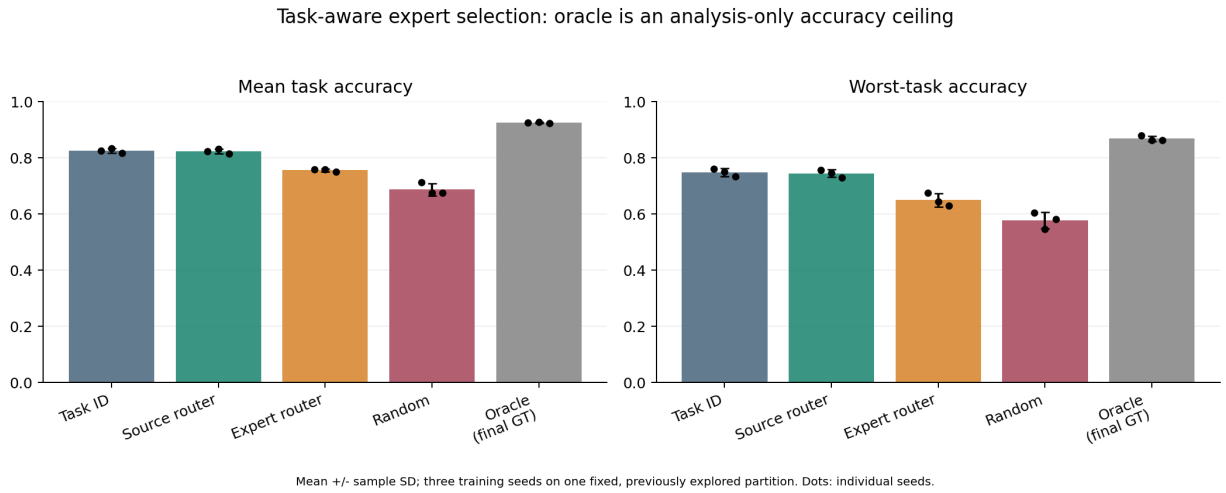


Figure 4: Task-aware routing analysis. Oracle uses final labels and is an accuracy ceiling, not an achievable serving result. Expert caches are used for offline analysis; their evaluation time is not a measurement of a deployed router.

We report the fractions of inputs helped and harmed relative to the own-task expert. Their difference equals the accuracy change, providing a direct interpretation of routing decisions. Source accuracy describes how often a route agrees with task identity; it does not establish that the choice improves prediction. The expert router additionally uses fit class labels and known task information, which must be included in its information budget.

5.6 Follow-up: conservative gates and cleaned official validation

The retained validation counts are 872, 408, 277, and 1023 for SST-2, MRPC, RTE, and CoLA. The split was previously explored historically; full-tuple overlap filtering does not remove all near duplicates or pretraining contamination.

Table 6: Locked methods on cleaned official GLUE validation. Prior historical exploration remains disclosed; oracle uses final GT.

Method	Native score	Mean accuracy	Worst accuracy
Individual / task ID	0.7618 ± 0.0153	0.8225 ± 0.0164	0.7353 ± 0.0623
Average	0.5738 ± 0.0178	0.7111 ± 0.0033	0.5391 ± 0.0205
Task Arithmetic	0.5874 ± 0.0170	0.7025 ± 0.0063	0.5571 ± 0.0149
TIES	0.6007 ± 0.0095	0.7024 ± 0.0066	0.5223 ± 0.0290
DARE+TA	0.5952 ± 0.0277	0.6971 ± 0.0120	0.5387 ± 0.0578
CE FP32	0.7050 ± 0.0220	0.7803 ± 0.0174	0.6750 ± 0.0719
Selected aligned H	0.7106 ± 0.0207	0.7807 ± 0.0179	0.6703 ± 0.0662
Selected shuffled H	0.7107 ± 0.0208	0.7806 ± 0.0177	0.6703 ± 0.0662
Source router	0.7552 ± 0.0137	0.8178 ± 0.0153	0.7316 ± 0.0646
Expert router	0.6571 ± 0.0219	0.7565 ± 0.0167	0.6462 ± 0.0407
Confidence gate	0.7618 ± 0.0153	0.8225 ± 0.0164	0.7353 ± 0.0623
Benefit gate	0.7618 ± 0.0153	0.8225 ± 0.0164	0.7353 ± 0.0623
Oracle (final GT)	0.8819 ± 0.0165	0.9150 ± 0.0146	0.8437 ± 0.0503

On cleaned official validation, confidence and benefit gates change macro mean accuracy versus task ID by +0.000 and +0.000 percentage points. A selection constraint is not a guarantee on final inputs; the actual help/harm rates and fallback choices determine the interpretation.

Both gate families select always-own in all three seeds. The zero final gain records the selection

and tie rules preferring the available baseline over learned alternatives. Some strongly regularized benefit candidates satisfy the selection floors but tie the reference; fewer switches or the baseline tie preference decides between them. This is not successful learning of useful switches. It avoids the original expert router's harms by retaining the existing task-ID reference. This is not a new routing accuracy advantage.

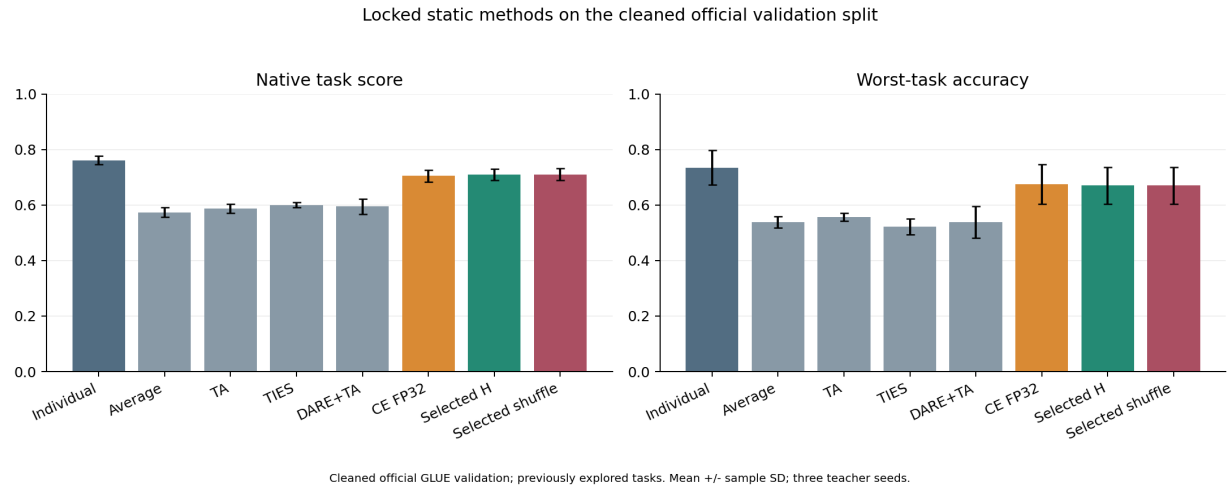


Figure 5: Locked static pipelines on cleaned official validation. Extra lambda candidates change the tuning budget. All methods use the known task head; tasks and the official split were historically explored.

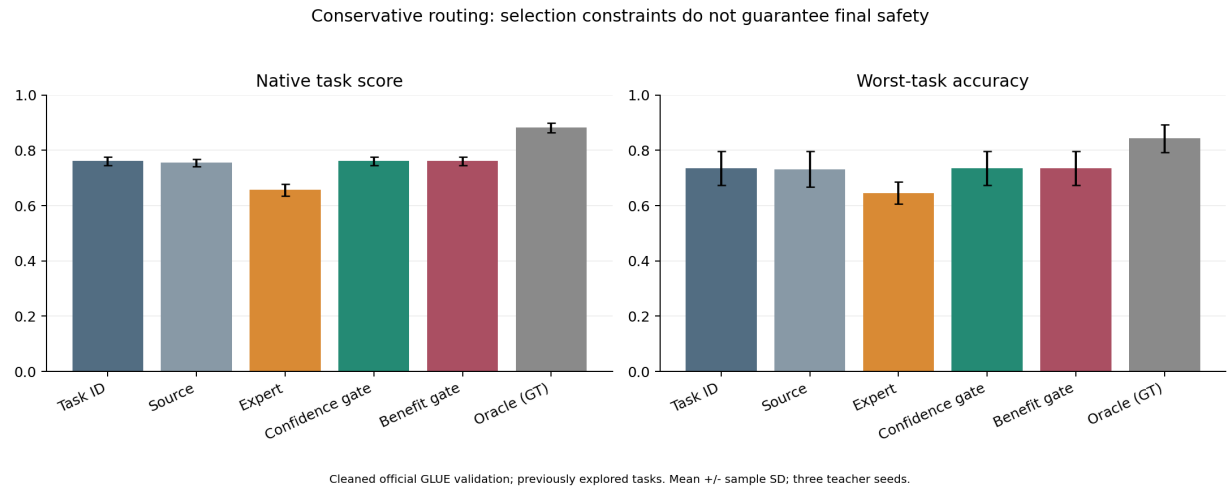


Figure 6: Actual final quality after conservative selection. A selection floor is not a final guarantee. Oracle uses final GT and is an analysis-only ceiling in this binary-task protocol.

6 Deployment and external-transfer diagnostics

6.1 Measured cost of the current implementation

An A100-SXM4-40GB benchmark uses fixed synthetic inputs, FP32, explicit CUDA synchronization, five warm-up calls and three rounds of 30 timed calls per shape/method. It includes base-feature extraction, CPU routing and state changes, and excludes model loading, tokenization, input transfer, and one-time materialization. These repeats are timing trials, not training seeds.

Table 7: Recorded implementation cost, batch one and 128 tokens. This benchmark uses the older seed-42 bank; it is separate from the v3 task-quality experiment.

Implementation	Median latency (ms)	Peak allocated (GiB)
Individual materialized	23.459	1.940
Layer-KL materialized	23.375	1.940
Router, CPU dense-delta switch	98.589	1.940
Router, GPU dense-delta cache	48.971	2.350

Static materialization has approximately the same single-backbone-forward cost as individual. Small submillisecond differences are not evidence of acceleration. Routing needs one base pass plus one forward for each distinct expert group in a batch; it is not always exactly two forwards. GPU caching reduces this implementation's transfer overhead but stores 336 MiB of dense deltas plus an 84 MiB zero-state copy. These costs are implementation-specific and do not characterize an optimized LoRA serving engine.

The new expert router's L4 quality and L4 serving latency have not been jointly benchmarked. We therefore do not combine its accuracy with historical A100 milliseconds to claim a measured Pareto improvement. The extra feature pass and expert grouping remain structural considerations.

One FP32 base has approximately 1.840 GiB of parameter payload. Four adapter directories total approximately 8.329 MiB, including their heads/configuration. Ninety-seven trained scalars are extra fitting parameters, not the entire deployment footprint. Comparing with four duplicated full backbones would exaggerate the storage saving relative to the actual shared-base bank.

6.2 Source cues and out-of-domain failures

A frozen earlier source router recognized 97.27% of 512 validation sources. Removing explicit field headers reduced this to 93.75%; uniform headers gave 95.12%. Titles contribute, but sentence structure, length, and dataset style remain possible cues. The edit changes positions and length and does not isolate every source confound.

An earlier fixed transfer check evaluated 512 IMDb and 512 PAWS examples per method, with three protocol-v2 training seeds and no target tuning.

Table 8: Historical protocol-v2 fixed transfer check: accuracy mean and sample SD across three training seeds; 512 examples per target.

Method	IMDb	PAWS
Individual	0.7090 ± 0.0237	0.4147 ± 0.0011
Average	0.6758 ± 0.0276	0.4258 ± 0.0135
Uniform gain 2	0.7116 ± 0.0118	0.4316 ± 0.0220
Layer CE	0.7051 ± 0.0198	0.4167 ± 0.0074
Layer KL (teacher-selected)	0.6953 ± 0.0237	0.4167 ± 0.0041

IMDb has substantial truncation at the 128-token limit; approximately 87% of those samples exceed it. PAWS methods, including individual, fall below the subset majority-class accuracy of 59.375%. These results do not establish robust transfer and cannot isolate merging as the sole cause. They involve earlier teachers, so they are not presented as new-v3 OOD measurements.

7 Discussion and limitations

The proposal's broad hypothesis contains distinct claims: similarity as a safety predictor, functional targets as a fitting signal, and dynamic routing as a useful serving mechanism. A successful implementation or high source-recognition score does not jointly establish these claims. The fixed v3 experiment supports learning a small static merge under calibration access, while its CE/KL control narrows the interpretation of that gain. The extension results further separate input-response matching from the distribution-preserving shuffle control, and expert identity prediction from actual help or harm relative to task-ID selection.

The strongest practical reference is often task-ID selection because the evaluation already knows the head. Static merging can simplify weight-state management, but it incurs a task-quality gap and offers limited total-storage savings for a small shared-base bank. Routing must justify its quality gain against the extra feature pass, expert grouping, and fitting information. The oracle clarifies what this bank could achieve with impossible knowledge; it does not demonstrate that the corresponding choices can be learned from the available features.

This is a small-backbone, four-task case study. The tasks are binary, share a fixed split, and have been explored before. Exact-tuple deduplication has a limited scope. Three training seeds do not support broad statistical conclusions. The selection labels make the pipeline supervised; stronger data-free or unknown-task claims require different protocols. We do not test every adapted-module subset, model family, LoRA rank, or modern alignment baseline. CE+H weight one and the energy floor are fixed design choices, not established optima. The shuffled control tests input pairing and leaves broader style confounding possible.

The original merging study has a conceptual connection to the CSA paper selected for Part 1. Section 8 strengthens that connection with direct reuse of the CSA implementation on expert-response matrices. It compresses functional-response objects, rather than LoRA weight updates, and retains separate reconstruction and downstream-utility checks.

8 Direct connection to the selected CSA paper

Our selected Part 1 paper proposes concurrent mode-wise subspaces for tensor-object reconstruction and recognition [1]. Its projection/reconstruction equations, conditional eigenspace updates and Kronecker-constrained view of PCA provide a concrete mechanism to transfer. We add an exploratory experiment that directly calls the unchanged Part 1 `TensorCSA` implementation on LoRA functional responses. This extends the original project; it does not retrospectively turn the earlier merging algorithms into CSA.

8.1 From image matrices to expert-response matrices

For input x, we stack the four experts' base-subtracted, masked-mean final hidden responses into $X(x) \in \mathbb{R}^{4\times 896}$. Expert identity and hidden coordinate are the two sample modes; the input-example axis is not a mode. No arbitrary reshaping of one vector into an image is used. With the fit mean M, CSA learns orthonormal $U \in \mathbb{R}^{4\times r_E}$ and $V \in \mathbb{R}^{896\times r_H}$:

$$C(x) = U^\top (X(x) - M)V, \qquad \widehat{X}(x) = UC(x)V^\top + M.$$

The objective minimizes summed squared reconstruction error on the 512 fit examples. As in the selected paper's conditional updates, each mode is optimized with the other fixed. We reuse covariance initialization and an energy-based stopping rule from our Part 1 implementation, with at most 30 sweeps. These implementation conventions differ from the original paper's initialization/stopping details; convergence is not a global-optimum claim.

This is the same structured-representation mechanism as Part 1, now applied to a different tensor object. Hidden coordinates are shared by homologous backbones; successful separability is tested rather than assumed. We evaluate all seven predeclared rank pairs. The 512 fit, 512 selection and 1,024 internal-final examples per training seed match the existing role IDs. Bases are learned without fit labels; final responses are collected after router choices lock. All three teacher seeds are reused, so this is not three additional teacher replications or a pristine blind evaluation.

8.2 Storage fairness and task usefulness

For $E = 4$, $H = 896$ and $N = 512$ cached objects, the full representation counts are

$$S_{\mathrm{CSA}} = EH + Er_E + Hr_H + Nr_E r_H, \qquad S_{\mathrm{PCA}} = EH + EHd + Nd.$$

The mean is included in both. Arrays use float64, so scalar ratios compare like-for-like numerical storage; they are not ZIP compression ratios. We report both $d = r_E r_H$ and the largest integer PCA rank that fits within each CSA budget. Unused integer-budget remainder is retained. Same-latent PCA has a more general projection and is checked to have no higher fit reconstruction error. This reproduces Part 1's distinction between latent size and storage budget.

For downstream utility, each representation feeds a standardized logistic expert selector with known-task one-hot features. Fit targets are minimum-ground-truth-NLL experts; $C \in \{0.1, 1, 10\}$ uses selection native score only. Raw delta and base features receive the same router procedure, while task-ID remains a directly available reference. This is supervised, offline probing: constructing $X(x)$ requires all four expert forwards. Neither representation-cache savings nor router quality demonstrates faster deployment or compression of LoRA weights.

Table 9: Direct CSA transfer to expert-by-feature response matrices. Storage includes a fit mean, bases and 512 codes in float64. Reconstruction errors are relative squared errors on the previously explored internal final. Routing requires all four experts. Mean ± sample SD over three fixed teacher seeds.

Representation	Latent	KiB	Rel. error	Native score
csa2x8	16	148.1	0.4848	0.7080 ± 0.0250
pca16	16	540.0	0.3843	0.7171 ± 0.0190
pca3	3	124.0	0.5785	0.6966 ± 0.0646
raw_delta	–	–	–	0.6729 ± 0.0023
base	–	–	–	0.6559 ± 0.0098
task_id	–	–	–	0.7695 ± 0.0114

For the illustrative $(r_E, r_H) = (2, 8)$ setting, CSA stores 148.1 KiB, versus 540.0 KiB for equal-latent PCA, a 3.65-fold reduction in this representation scope. The largest budget-feasible PCA rank is 3. Internal-final relative squared errors are 0.4848 (CSA), 0.3843 (equal-latent PCA), and 0.5785 (budget PCA). Their native routing scores are 0.7080, 0.7171, and 0.6966, respectively; raw-delta routing is 0.6729 and task-ID is 0.7695. These comparisons separate representation compactness, reconstruction fidelity and task utility. The full predeclared grid is available in the evidence CSV; this illustration is not a final-selected best configuration.

The defensible link is now operational: Part 1 explains and tests structured tensor subspaces; Part 2 applies the same implementation to multi-adapter response objects and checks whether compact reconstruction preserves information useful for expert selection. The merge-weight study and this response-compression study answer complementary questions. We do not claim a new CSA algorithm, a CSA-based weight merger, or a free router.

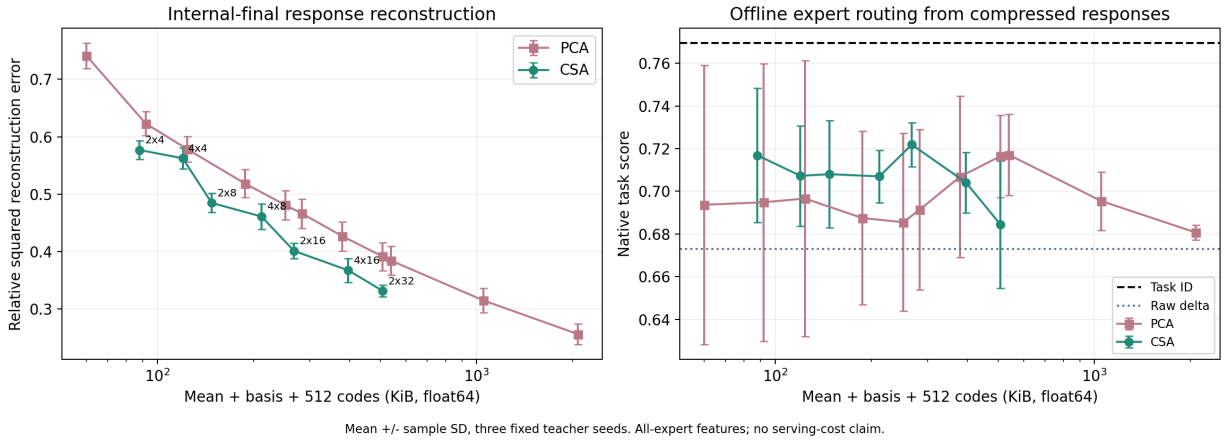


Figure 7: The direct CSA bridge: storage versus reconstruction and downstream information. Error bars are sample standard deviations over the same three teacher seeds. The storage scope excludes base/adapters, logits, router state and serialization.

8.3 Discriminative learning and task-ID residual gates

The selected CSA paper also evaluates discriminative processing after tensor representation [1]. We therefore test whether regularized LDA can improve expert selection from the frozen CSA cores. CSA bases are unchanged and remain reconstruction-trained; LDA changes the downstream supervised objective. All seven CSA ranks, corresponding PCA controls, raw delta and base features use identical new method grids. This is an exploratory follow-up to inspected final results.

LDA uses empirical class priors and shared covariance

$$\Sigma_\alpha = (1-\alpha)S_w + \alpha \frac{\mathrm{tr}(S_w)}{D} I, \qquad \alpha \in \{0.1, 0.5, 0.9\}.$$

Its linear discriminant is implemented with a dual solve for high-dimensional raw features and checked against the standard scikit-learn shrinkage-LDA implementation. A small diagonal floor handles zero within-class variance. Scaling uses fit data only; known-task one-hot features are then appended.

We compare three rules. *Direct LDA* predicts the minimum-ground-truth-NLL expert. *Task prior* keeps the own expert unless an alternative's predicted probability exceeds the own expert's probability by a selection-chosen margin. *Benefit LDA* predicts help, tie or harm relative to the own expert for each alternative, and switches only if $P(\text{help}) - P(\text{harm})$ exceeds a chosen threshold. An expert's own-task examples have identically zero benefit, so they are excluded from its benefit-classifier fit; each such classifier uses 384 examples. These posteriors are model estimates, not established calibrated guarantees.

Direct LDA chooses shrinkage by selection native score. Both gated families include always-own and require selection native and macro accuracy at least task ID, with worst accuracy no more than 0.01 lower. Among eligible candidates we maximize native, then macro accuracy, then minimize switching; ties prefer always-own and stronger regularization. These are selection constraints rather than guarantees on final data. All choices lock before reading the internal final. Every candidate's selection routes and the selected final routes are independently reloaded and recomputed.

At the fixed CSA 2×8 setting, direct LDA has native score 0.7485, versus 0.7080 for the earlier logistic router and 0.7695 for task-ID selection.

The task-prior gate has native score 0.7695; it chooses always-own in 3/3 seeds and makes nonzero final switches in 0/3. Mean switched, helped and harmed fractions are 0.000%, 0.000%, and 0.000%, respectively. Its macro-accuracy change versus task ID is +0.000 percentage points.

Table 10: Discriminative follow-up on the same internal final, three fixed teacher seeds. All candidates use matched grids across representations. Switch/help/harm are mean percentages; native score is mean ± sample SD. Own is the number of seeds selecting always-own.

Features	Method	Native	Switch	Help	Harm	Own
task_id	reference	0.7695 ± 0.0114	0.00	0.00	0.00	3/3
csa2x8	previous_logistic	0.7080 ± 0.0250	19.43	1.30	3.71	0/3
csa2x8	direct_lda	0.7485 ± 0.0290	11.49	0.65	1.20	0/3
csa2x8	task_prior	0.7695 ± 0.0114	0.00	0.00	0.00	3/3
csa2x8	benefit_lda	0.7695 ± 0.0114	0.00	0.00	0.00	3/3
pca16	benefit_lda	0.7652 ± 0.0177	0.39	0.07	0.20	2/3
pca3	benefit_lda	0.7662 ± 0.0161	0.42	0.03	0.13	2/3
raw_delta	benefit_lda	0.7695 ± 0.0114	0.00	0.00	0.00	3/3

The benefit-LDA gate has native score 0.7695; it chooses always-own in 3/3 seeds and makes nonzero final switches in 0/3. Mean switched, helped and harmed fractions are 0.000%, 0.000%, and 0.000%, respectively. Its macro-accuracy change versus task ID is +0.000 percentage points.

These outcomes distinguish discriminative fitting, conservative fallback and actual useful switching. Matching the reference with zero switches is baseline preservation rather than evidence of learned routing benefit. Selection constraints alone do not establish final safety or a statistically significant improvement.

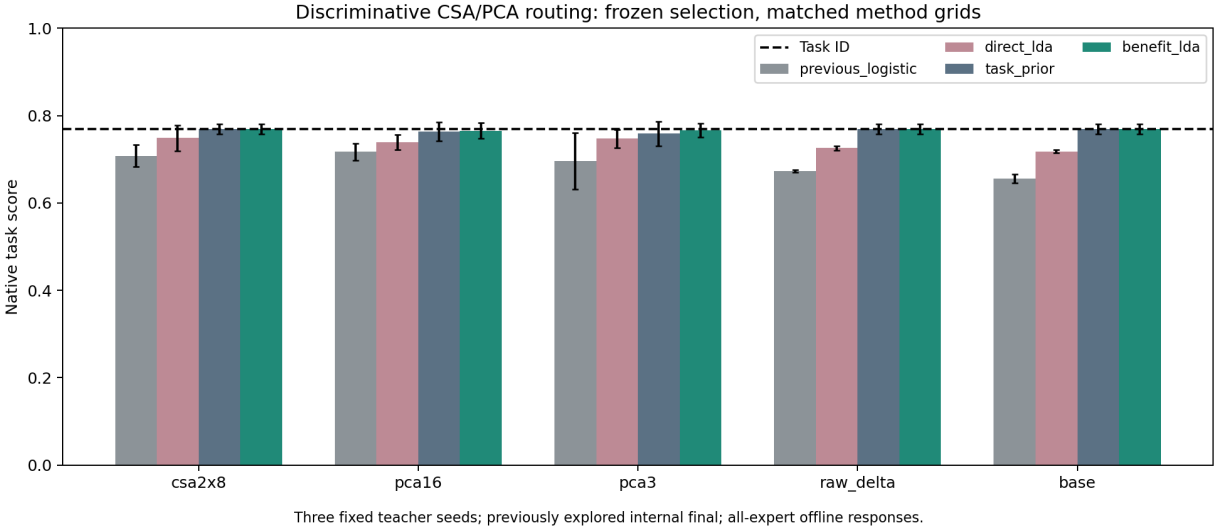


Figure 8: Discriminative and residual-gating controls on fixed representations. Error bars show sample SD across the same three teacher seeds. Matching task ID by selecting always-own is distinguished from learning useful switches; actual help/harm records accompany the results.

All-expert responses are still required. Thus any improvement here concerns offline information use, not reduced serving cost. The native score combines task-specific metrics; helped minus harmed equals the macro-accuracy change, rather than the native-score change. We report the full frozen rank grid and do not replace the illustrative rank by a final-selected winner.

9 Conclusion

We implemented a reproducible, task-aware LoRA merging study with weight-space baselines, supervised and functional coefficient objectives, explicit data boundaries, and routing diagnostics. The v3 mean native task scores are 0.7695 for individual, 0.7132 for layer CE, 0.6987 for layer KL, and 0.6146 for our TIES variant. Learned coefficients recover part of the averaging loss, while teacher-output KL has no average advantage over its matched-capacity CE control. The direct

hidden and task-aware routing extensions report paired changes of -0.0733 native score and -6.93 accuracy points against their respective controls, with the stated exploratory scope. The larger cleaned-validation follow-up gives selected hidden-minus-CE native change +0.0055 and benefit-gate-minus-task-ID accuracy change +0.00 points. These results describe a selected pipeline under known-task access, not a new blind benchmark or equal-budget mechanism proof. The defensible outcome is evidence about the utility and failure conditions of functional proxies in this setting, together with measured implementation trade-offs and a reproducible pipeline.

A Reproducibility and implementation checks

After a browser interruption, the earlier temporary VM was reclaimed before complete result download. We recovered intact CSV and figure records from the partial archive, verified their CRC and original manifest hashes, and reran the frozen extension on 11 November from the checksummed 12-teacher backup. The final package contains the full rerun predictions and a comparison with recovered metrics. This repeats the same three training seeds and does not increase the number of independent replications to six.

Exact model/data revisions and partition checksum are recorded below.

> Model: `060db6499f32faf8b98477b0a26969ef7d8b9987`
> GLUE: `bcdcba79d07bc864c1c254ccfcedcce55bcc9a8c`
> Partition SHA256:
> `526c3ae3aefebf36f651c0518d4a69b1c51d8c2e9d5145063f791b0b363386eb`

The code records environment versions, ID lists, selected checkpoints, coefficients, gains, source SHA, and prediction artifacts. Model computation runs in Colab; generated sources and figures accompany this report.

The pinned research stack uses a system-site-packages virtual environment. Inherited optional packages generated pip dependency-conflict warnings; those unrelated packages are not used here. We report successful research-module checks and actual imports rather than a globally clean pip check.

Checks cover merge arithmetic, masked pooling, a shuffled target's lack of fixed points, oracle invariance to additive logit shifts, nonzero coefficient gradients, no gradients on frozen parameters, low-rank/dense logit parity, task-ID/individual metric parity, and split permissions. The extension additionally audits saved predictions independently of model fitting. Versioned results preserve exploratory stages instead of overwriting earlier outcomes.

The v3 checkpoint backup contains 12 adapters and 131 verified files. Restoration was tested into a new directory with relocated manifest paths; corrupt SHA, unsafe archive paths, and overwriting an existing directory were rejected. The shared base/tokenizer are obtained separately at pinned Hub revisions. The runtime connection client is closed after exports have been downloaded and verified; closing it is distinct from releasing the cloud VM.

The follow-up checks cover scalar-weight overrides, fallback behavior without feature extraction, selection rejection and tie rules, label-blind content filtering, stored classifier probabilities, selection-only lambda choice, cached split/label alignment, and reload parity of the selected gates. Saved official predictions are audited independently for every method and task. All experiments use frozen teacher banks; feature caches and raw text stay outside the published evidence archive.

A.1 Execution overview

```
python code/run_independent_study.py --output outputs/new-v3
python code/independent_baselines.py --study outputs/new-v3
```

```
python code/activation_extension.py --study outputs/new-v3 \
  --plan configs/activation-strengthening-v1.json \
  --output outputs/new-extension
```

Output directories must be new. The extension requires the matching locked manifest and teacher bank. Compiler commands and data-generation sources are provided in the package README. Group scheduling, formal submission requirements, and teammate access are administrative checks separate from scientific completion.

References

[1] D. Xu et al. *Reconstruction and Recognition of Tensor-Based Objects With Concurrent Subspaces Analysis*. IEEE TCSVT, 18(1):36–47, 2008. https://doi.org/10.1109/TCSVT.2007.903317.

[2] E. J. Hu et al. *LoRA: Low-Rank Adaptation of Large Language Models*. 2021. https://arxiv.org/abs/2106.09685.

[3] G. Ilharco et al. *Editing Models with Task Arithmetic*. ICLR 2023. https://arxiv.org/abs/2212.04089.

[4] P. Yadav et al. *TIES-Merging: Resolving Interference When Merging Models*. 2023. https://arxiv.org/abs/2306.01708.

[5] L. Yu et al. *Language Models are Super Mario: Absorbing Abilities from Homologous Models as a Free Lunch*. ICML 2024. https://arxiv.org/abs/2311.03099.

[6] X. Jin et al. *Dataless Knowledge Fusion by Merging Weights of Language Models*. ICLR 2023; arXiv version 6 consulted. https://arxiv.org/abs/2212.09849.

[7] E. Yang et al. *AdaMerging: Adaptive Model Merging for Multi-Task Learning*. ICLR 2024. https://arxiv.org/abs/2310.02575.

[8] G. Stoica et al. *Model Merging with SVD to Tie the Knots*. 2024 preprint / ICLR 2025. https://arxiv.org/abs/2410.19735.

[9] A. Romero et al. *FitNets: Hints for Thin Deep Nets*. 2015. https://arxiv.org/abs/1412.6550.

[10] A. Wang et al. *GLUE: A Multi-Task Benchmark and Analysis Platform for Natural Language Understanding*. 2018. https://arxiv.org/abs/1804.07461.

[11] Qwen Team. *Qwen2.5 Technical Report*. 2024; revised 2025. https://arxiv.org/abs/2412.15115.